

Interfaces in Object-Oriented Programming

A Beginner's Guide

The sequel to “Polymorphism and Inheritance”

Zadid Bin Azad

Department of Computer Science and Engineering

Islamic University of Technology

zadidbinazad@iut-dhaka.edu

July 22, 2026

Teaching Insight

This guide continues the story of the previous one and assumes its ending. There we created *subtypes* using `extends`, and found that runtime polymorphism only ever needed the *promise* that an object understands a call — not the inherited code that came bundled with it. That guide closed on a single question: can we make the promise, and create subtypes, *without* a parent class handing down any implementation? This guide is the answer. As before, a few boxes are titled **Question to Carry Forward**; they are *not* answered on the page. Each carries a label such as (Q1) and one clue. Attempt each yourself; the reasoning in that gap is the point.

1 Where We Left Off

The previous guide built a payroll system around an abstract `Employee` class. Every real employee was a `Manager`, a `Developer`, or a `Salesperson`; each supplied its own `calculatePay()` formula; and because all three were *subtypes* of `Employee`, a single loop over an `Employee[]` could pay the whole staff, the JVM dispatching to each object's own formula at runtime.

That machinery hinged on one guarantee: any object held in an `Employee` variable *understands the call* `calculatePay()`. Inheritance delivered that guarantee — but it delivered a great deal more besides. Along with the promise came shared fields (`name`, `id`, `baseMonthly`), shared code (`describe()`, `toString()`), a constructor to chain through, and a claim of *identity*: a `Manager` genuinely *is* a kind of `Employee`.

We are about to meet a business requirement where that bundle is exactly the wrong shape — where we need the promise and nothing else.

2 The Wall: Paying People Who Are Not Employees

The payroll department returns with a new fact about the world:

*The company does not only pay its own staff. Each month it also pays outside **contractors** (per signed-off milestone) and settles invoices from **vendors** (per delivered goods). Accounts want **one** monthly run that computes what is owed to everyone the company must pay — staff, contractors, and vendors — in a single pass.*

Take the sentence seriously, the way we always do: the load-bearing phrase is “everyone the company must pay ... in a single pass.” A single pass over a mixed collection is precisely

the polymorphic loop we already know how to write. So the instinct is immediate — make **Contractor** and **Vendor** subtypes of something, put them in one array, and loop.

The instinct then runs straight into a wall. What would that common parent be? It cannot be **Employee**: a contractor is emphatically *not* an employee (no salary, no `id` on the staff roll, different tax treatment, works for other clients too), and a vendor is not a person at all — it is a company. Forcing `class Contractor extends Employee` would be the exact “is-a with a straight face” failure the previous guide warned about: the type system would then believe a contractor has a staff `id` and a base monthly salary, which is a lie.

Question for the Class

We want **Employee**, **Contractor**, and **Vendor** to sit in one array and be paid in one loop — which requires a common supertype they can all be substituted for. Yet no honest *is-a* relationship links them: none is a kind of the others, and inventing a **PayableThing** class for all three to extend would force them to inherit fields and code none of them share. What, exactly, do these three actually have in common?

Working It Through

They share a *capability*, not an identity. Each one *can be asked*, “*how much do we owe you this month?*” — and each answers by its own rule. That is the entire commonality: one operation they can all perform. They do not share state, they do not share code, and they do not share a place in any family tree. Abstract-class inheritance is built to share all of those; here we must share *only the promise of one operation*. That is what an interface is.

3 What an Interface Is

An **interface** is a named contract: a list of method signatures that a class can promise to fulfil, carrying *no* instance state and (in its purest form) *no* implementation. It is a type made of promises alone. Where a class answers “what is this object, and how is it built?”, an interface answers only “what can this object be asked to do?”

```
1 public interface Payable {  
2     // No fields (no per-object state). No constructor. No bodies.  
3     // Just the promise of one operation, in BDT paisa.  
4     long amountOwedThisMonth();  
5 }
```

Listing 1: An interface is a pure contract: a name and a set of promises.

Three things are true of that tiny declaration, and each is worth saying aloud in class:

- **Every method is implicitly public and abstract.** You do not write those words; the interface writes them for you. `amountOwedThisMonth()` is a signature with a semicolon and no body — a pure “someone must supply this.”
- **There is no instance state and no constructor.** An interface cannot be `new`-ed and holds no per-object fields. (Any field you *could* write in it is silently `public static final` — a shared constant, not object state. More on that later.)
- **It is a *type*.** `Payable` can be used as the declared type of a variable, a parameter, a return value, or an array element — exactly like a class name. This is the property that will give us polymorphism.

A class signs the contract with the keyword `implements`, and must then provide a body for every promised method:

```

1 public class Employee implements Payable {           // still an abstract-class
    hierarchy...
    // ... name, id, baseMonthly, calculatePay(), etc. from before ...
    @Override
    public long amountOwedThisMonth() {
2         return calculatePay();                       // reuse the pay formula we already
3         have
4     }
5 }
6
7
8
9 public class Contractor implements Payable {         // a completely separate
    class
10    private final String firm;
11    private final long ratePerMilestone;              // BDT paisa
12    private final int milestonesSignedOff;
13
14    public Contractor(String firm, long ratePerMilestone, int
        milestonesSignedOff) {
15        this.firm = firm;
16        this.ratePerMilestone = ratePerMilestone;
17        this.milestonesSignedOff = milestonesSignedOff;
18    }
19    @Override
20    public long amountOwedThisMonth() {
21        return ratePerMilestone * milestonesSignedOff;
22    }
23 }
24
25 public class Vendor implements Payable {             // not even a person
26    private final String company;
27    private final long invoiceTotal;                  // BDT paisa
28
29    public Vendor(String company, long invoiceTotal) {
30        this.company = company;
31        this.invoiceTotal = invoiceTotal;
32    }
33    @Override
34    public long amountOwedThisMonth() {
35        return invoiceTotal;
36    }
37 }

```

Listing 2: Three unrelated classes, each signing the same contract its own way.

Notice what did *not* happen. `Contractor` and `Vendor` did not inherit a single field or line of code from `Payable` — there was nothing to inherit. They kept their own unrelated shapes: a contractor has a firm and milestones, a vendor has an invoice total, an employee has everything it had before. All three merely made the *same promise*, and each honoured it in its own way.

Teaching Insight

Put the three classes on the projector side by side and ask: “What field do these classes share?” The answer is *none*. Then ask: “What method do they share?” The answer is *one signature, three different bodies*. That contrast — zero shared state, one shared promise — is the whole reason interfaces exist, and it is visible in the source without running anything.

4 Subtyping With an Interface

Here is the payoff, and it is the same payoff `extends` gave us — reached by a different road. **When a class implements an interface, the class becomes a subtype of that interface.** A `Contractor` *is a* `Payable`; a `Vendor` *is a* `Payable`; an `Employee` *is a* `Payable`. Therefore a variable declared as `Payable` may legally hold any of them, and every one of them is guaranteed to understand any call the `Payable` contract promises.

That is precisely the two-part condition the previous guide identified as the prerequisite for runtime polymorphism — one variable able to refer to objects of different classes, all of them guaranteed to understand a shared call. Interfaces satisfy it without a shared parent class.

```
1 Payable[] toPay = {
2     new Employee("Ada", 1, 9000_00, /* ... */),    // an Employee
3     new Contractor("BrightBuild", 40000_00, 3),    // a Contractor
4     new Vendor("PaperSupply Ltd", 12500_00),       // a Vendor
5     new Contractor("SoloDev", 25000_00, 1)
6 };
7
8 long grandTotal = 0;
9 for (Payable p : toPay) {                          // declared type is Payable for all four
10     grandTotal += p.amountOwedThisMonth();         // each runs ITS OWN rule
11 }
12 System.out.println("Owed this month (paisa): " + grandTotal);
```

Listing 3: One array, one call name, three unrelated classes paid in one sweep.

The variable `p` is declared `Payable`, so the compiler checks only that `Payable` promises an `amountOwedThisMonth()` — which it does. The compiler does *not* choose which body runs. On each iteration the JVM inspects the actual object — an `Employee` here, a `Vendor` there — and dispatches to *that class's* implementation. This is the identical late-dispatch mechanism from the inheritance guide; the only thing that has changed is *how the subtype relationship was created*. There, by inheriting code. Here, by promising a contract.

Question for the Class

In the loop, `p`'s declared type is `Payable`, which has no body for `amountOwedThisMonth()` at all — it is a bare signature. Yet `p.amountOwedThisMonth()` compiles and runs the right formula for each object. Separate two questions you have seen before: (a) why does the *compiler* accept the call, looking only at the declared type `Payable`? and (b) why does the *running program* execute the `Vendor` body on the vendor and the `Contractor` body on the contractor?

Working It Through

Exactly the same two-selector answer as for overriding. (a) The compiler reasons about the *declared* type and asks only “is this call promised by `Payable`?” — yes, so it compiles. (b) The JVM reasons about the *actual* object and asks “which class are you really?”, then runs that class's version. An interface method is abstract in the same way an abstract method is; dispatch does not care whether the promise arrived via `extends` or via `implements`.

Where Students Typically Slip

Students write `Payable p = new Payable();` and are baffled by the compile error. An interface is a contract, not a thing: it has no constructor and no state, so there is nothing to construct. You may only `new` a concrete class that *implements* `Payable`, and then *store* it in a `Payable` variable. The type on the left is the promise; the class on the right is the object that keeps it.

5 The Two Things an Interface Does That an Abstract Class Cannot

We could have made `Payable` an abstract class with one abstract method and gotten the paid-in-one-loop behaviour. So why does the language offer interfaces as a separate tool? Because two capabilities fall out of “a contract with no state” that a stateful parent class simply cannot provide.

5.1 One class can implement many interfaces

A class in Java may *extend exactly one* parent. That is a hard limit — Java has no multiple *class* inheritance, precisely to avoid the ambiguity of inheriting conflicting state and code from two parents. But a class may *implement any number* of interfaces, because a contract brings no state to conflict over. This lets an object honestly play several *roles* at once.

Suppose accounts also need, of some payees, that they can be taxed (staff and contractors are; foreign vendors may not be) and that they can produce an audit line for the ledger:

```

1 public interface Payable { long amountOwedThisMonth(); }
2 public interface Taxable { long taxWithheld(long grossPaiza); }
3 public interface Auditable { String auditLine(); }
4
5 // A contractor can be paid, is taxed, and must appear in the audit log:
6 public class Contractor implements Payable, Taxable, Auditable {
7     // ... fields as before ...
8     @Override public long amountOwedThisMonth() { return ratePerMilestone *
9         milestonesSignedOff; }
10    @Override public long taxWithheld(long gross) { return gross / 10; }
11    // 10% withholding
12    @Override public String auditLine() { return "Contractor:" + firm; }
13 }
14
15 // A foreign vendor is paid and audited, but NOT taxed here -> it simply omits
16 // Taxable:
17 public class ForeignVendor implements Payable, Auditable {
18     @Override public long amountOwedThisMonth() { /* ... */ return 0; }
19     @Override public String auditLine() { return "ForeignVendor"; }
20 }

```

Listing 4: Separate roles as separate contracts; a class opts into the ones it plays.

Now each contract can drive its own loop over exactly the objects that satisfy it, and a single object can be viewed through whichever role a piece of code cares about:

```

1 Contractor c = new Contractor("BrightBuild", 40000_00, 3);
2 Payable    asPayable    = c;    // view it as "something we pay"
3 Taxable    asTaxable    = c;    // view it as "something we tax"
4 Auditable  asAuditable  = c;    // view it as "something we log"

```

```
5 // Three views, one object. Each view exposes only that role's promised methods
   .
```

Listing 5: The same object, seen through different role-types.

Question for the Class

A **Manager** is one kind of thing — an **Employee** — and can have only one such identity, which is why **extends** is limited to one parent. But a **Contractor** can be *several things it is able to do at once*: payable, taxable, auditable. Why is it safe for a class to promise many contracts, when it is unsafe (and forbidden) for a class to inherit from many parent classes?

Working It Through

Because a contract carries no state or code to collide. If a class could inherit from two parent classes that each defined a field **balance** and a method **save()**, the compiler would face an unanswerable question: *which balance, whose save()*? That is the classic “diamond” ambiguity. Interfaces (in their pure form) declare only signatures — there is no state to duplicate and no body to choose between, so stacking many of them raises no ambiguity. Many *promises* compose cleanly; many *implementations* do not.

5.2 Unrelated classes can share one interface

An abstract class can only lend its type to its *descendants* — classes willing to sit beneath it in one family tree. An interface has no such requirement. **Employee** lives in a rich inheritance hierarchy; **Vendor** descends from nothing; **Contractor** is its own thing. They occupy entirely different regions of the type system, yet all three are **Payable**. The interface cuts *across* hierarchies, grouping objects by what they *can do* rather than by what they *are*.

This is why the standard library models capabilities as interfaces: anything at all can be **Comparable** (numbers, dates, employees, invoices), anything can be **Iterable**, anything can be **Runnable**. None of those force you into a particular base class. You bolt the capability onto whatever class already exists.

Teaching Insight

A one-line summary that sticks: *inheritance groups things by what they **are**; interfaces group things by what they **can do***. A **Manager** and a **Developer** belong together because they *are* employees. A **Contractor** and a **Vendor** belong in the pay run because they *can be paid* — nothing more.

6 “Is-a” Versus “Can-do”: the Language Test

The previous guide gave a spoken test for choosing between **extends** and a field: say “B is a A” versus “B has a A.” Interfaces add a third phrasing, and sharpen the choice into a clean three-way decision:

- “B is a A” (a **Manager** is an **Employee**) — and they share state and code — points to **extends**: class inheritance.
- “B is able to A” / “B can act as A” (a **Contractor** is able to be paid; a **Vendor** can act as a payee) points to **implements**: an interface. The relationship is a *role* or *capability*, not a family identity.

- “**B has a A**” (an `Employee` has an `Address`) points to a **field**: composition. Unchanged from before.

A good name for an interface therefore often ends in `-able` or reads as a role: `Payable`, `Taxable`, `Comparable`, `Runnable`, `Drawable`. The suffix is a reminder that the type names an *ability*, not a *kind*.

Where Students Typically Slip

Students make an interface named after a *noun/thing* (interface `Animal`) when they mean a kind-of hierarchy with shared state, or make an *abstract class* named after an *ability* (abstract class `Comparable`) and then cannot apply it to a class that already has a parent. Match the tool to the sentence: a shared *identity* with shared state wants an (abstract) class; a shared *ability* that any class might possess wants an interface.

Question to Carry Forward (Q1)

Consider `Comparable`: numbers, dates, strings, and our `Employee` can all be compared, and none of them share a common base class devoted to comparison. Suppose the Java designers had instead shipped comparison as an abstract class `Comparable` that types must **extend**. What could a class that already **extends** something else — say `Employee` **extends** `Person` — no longer do, and why does making `Comparable` an *interface* remove that obstacle entirely?

Clue: count how many classes a Java class may **extend**, and how many interfaces it may **implement**. One of those numbers is the whole answer.

7 Interface Versus Abstract Class, Head to Head

Both are types you cannot instantiate directly; both can hold abstract methods that subtypes must fill in; both give you subtypes and therefore runtime polymorphism. The differences are about *what else* each one carries.

	Abstract class	Interface
Keyword to subtype	extends	implements
How many can a class take?	One parent only	Many interfaces
Instance state (fields)	Yes — can hold per-object fields	No per-object fields; only public static final constants
Constructor	Yes (run via super in subclasses)	None
Method bodies	Yes, freely (concrete methods)	Only default/static/private methods (see next section); the rest are abstract
Models the relationship	“ <i>is a kind of</i> ” (identity)	“ <i>is able to / acts as</i> ” (capability, role)
Reaches across hierarchies?	No — only its descendants	Yes — any class, anywhere
Access modifiers on members	Any (private , protected , ...)	Members are public (methods)

The choice, boiled down:

- Reach for an **abstract class** when the subtypes genuinely *are a kind of* one thing and you have real shared *state* and *code* to hand down — our `Employee` with its `name`, `id`, `describe()`, and validating constructor. One family, shared substance.
- Reach for an **interface** when you are naming an *ability* that classes from anywhere might

possess, when you need a class to play *several* roles at once, or when there is no shared state to justify a common parent — our `Payable` across employee, contractor, and vendor.

Teaching Insight

The two are not rivals; the richest designs use both. `Employee` *extends* an abstract base for its shared salary machinery *and implements* `Payable` to join the cross-cutting pay run. Show that one class doing both on the same line — `class Employee extends StaffMember implements Payable, Taxable` — so students see that “is a kind of” and “is able to” are answered independently, by different clauses of the same declaration.

Question to Carry Forward (Q2)

Since Java 8 an interface may carry **default** methods *with* bodies, which looks a lot like an abstract class's concrete methods. So could we replace every abstract class with an interface? Try it on `Employee`: attempt to move its `name`, `id`, and `baseMonthly` *fields*, and its validating *constructor*, into an interface. What stops you, and what does that tell you about the one thing an abstract class will always be able to do that an interface never can?

Clue: the blocker is not about methods at all. Ask what an interface is forbidden to hold, and what a constructor exists to initialise.

8 What Modern Interfaces May Also Contain

Early Java interfaces were *only* signatures and constants. Modern Java (the JDK 17 you are using) allows an interface to carry a few kinds of body as well. Each was added to solve a specific problem, and it is worth knowing *why* each exists, not just that it does.

8.1 Constants

Any field written in an interface is implicitly `public static final` — a shared, unchangeable constant belonging to the interface itself, not to any object. It is not per-object state; it is a named value all implementers (and all callers) can refer to.

```

1 public interface Payable {
2     long MIN_PAYOUT_PAISA = 100_00;    // implicitly public static final; == BDT
3     long amountOwedThisMonth();
4 }
5 // Referred to as Payable.MIN_PAYOUT_PAISA anywhere.
```

Listing 6: Interface constants are public static final by default.

8.2 default methods: giving the contract a body

A **default** method is a method *with* an implementation, provided by the interface itself, that every implementer receives for free but may override. It was added for one honest reason: **to let an interface grow without breaking the classes that already implement it.**

Picture the problem. Thousands of classes across the codebase implement `Payable`. You now want every payable to also report a human-readable receipt line. If you add a new *abstract* method `receiptLine()` to the interface, every one of those thousands of classes instantly fails to compile until it supplies a body — a catastrophe. A **default** method threads the needle: it

adds the method *and* a sensible fallback body, so old implementers keep compiling while new ones may override for something better.

```

1 public interface Payable {
2     long amountOwedThisMonth();           // still abstract: everyone
        must supply this
3
4     default String receiptLine() {         // NEW, with a fallback body
5         return "Owed: " + amountOwedThisMonth() + " paisa";
6     }
7 }
8 // Every existing Payable class compiles unchanged and gets receiptLine() for
   free.
9 // A class that wants a nicer receipt simply overrides it.

```

Listing 7: A default method: new behaviour that does not break existing implementers.

Where Students Typically Slip

Students start treating **default** methods as a licence to put ordinary shared code in interfaces and abandon abstract classes. Resist this. A **default** method still cannot touch per-object *fields* (there are none) and cannot be built by a constructor. Its proper jobs are narrow: supplying a convenience method defined purely in terms of the other contract methods, or evolving an interface safely. Real shared *state* still belongs in a class.

Question for the Class

Two interfaces, `Payable` and `Refundable`, each declare a default `String note()` with a different body. A class implements *both*. When you call `note()` on an object of that class, whose default runs — and how could the compiler possibly know?

Working It Through

It cannot know, and Java refuses to guess. This is the diamond ambiguity resurfacing — but only because both defaults carry *bodies*. Java's rule: when a class inherits two **default** methods of the same signature, the class *must* override `note()` itself to resolve the clash (it may delegate explicitly with `Payable.super.note()` to pick one). Note that this danger exists *only* for methods with bodies; two *abstract* promises of the same signature collapse harmlessly into one. Behaviour is what creates ambiguity, which is exactly why pure signatures compose freely and implementations do not.

8.3 static and private methods

An interface may also hold **static** methods — utilities that belong to the contract as a whole rather than to any instance (often factory-style helpers, e.g. `Payable.of(...)`) — and, since Java 9, **private** methods, which exist only to share code *between the interface's own default methods* without exposing it to the world. Both are conveniences that keep contract-related helper code next to the contract; neither adds per-object state.

8.4 Functional interfaces and lambdas

An interface with *exactly one* abstract method is called a **functional interface**. It is special because Java lets you supply its single implementation with a compact *lambda* expression instead of a whole class. You have effectively met this idea already; here it is made explicit.

```

1 @FunctionalInterface                                // optional; asks the compiler to enforce
   "exactly one"
2 public interface PayRule {
3     long compute(long basePaisa);    // the single abstract method
4 }
5
6 PayRule flat    = base -> base;        // the lambda IS the
   implementation
7 PayRule bonus   = base -> base + 2000_00; // another implementation, no
   named class needed
8
9 long p = bonus.compute(9000_00);        // dispatches to the second lambda
   's body

```

Listing 8: A functional interface can be implemented by a lambda.

The optional `@FunctionalInterface` annotation asks the compiler to *verify* there is exactly one abstract method, so you find out immediately if someone later adds a second and accidentally breaks lambda usage. The deep point for the lecture: a lambda is just a very small, anonymous implementer of a one-method contract — the same subtype-and-dispatch story, written tersely.

8.5 Marker interfaces

An interface with *no* methods at all — `java.io.Serializable` is the classic — is a **marker**. It promises no operation; it exists purely so that other code can ask “is this object tagged with that capability?” (via `instanceof`) and behave accordingly. It is the minimal interface: a type that carries only a yes/no fact about the class.

9 An Interface Can Extend Other Interfaces

Interfaces relate to each other with `extends` (not `implements`), and here Java’s one-parent limit does *not* apply — an interface may extend *several* others, because, once again, there is no state to collide. The result is a larger contract: implementing the child obliges you to fulfil everything it and its parents promise.

```

1 public interface Settlement extends Payable, Auditable {
2     // adds nothing new of its own, but a Settlement is anything that is
3     // BOTH payable AND auditable. Implement Settlement -> owe both methods.
4     String settlementReference(); // ... and one more of its own
5 }

```

Listing 9: A composite contract built from smaller ones.

This lets you assemble fine-grained capabilities into named bundles, and it mirrors, at the contract level, the multi-role composition we saw at the class level.

10 How Interfaces Create Abstraction

Everything so far has been about *subtyping* — letting one variable stand for many kinds of object. The final idea is the one interfaces are most prized for in real design: **abstraction**, in the precise sense of *separating what something does from how it does it*, so that code which uses a capability need never know — and never depends on — who provides it.

Return to the monthly run. Suppose accounts want the total owed, written wherever they choose: today to the console, tomorrow to a printed report, next month to a database. Write the code that *needs* an output against a contract, not against any concrete writer:

```

1 public interface LedgerSink {           // "somewhere a settlement line can go"
2     void write(String line);
3 }
4
5 public class PayrollRun {
6     // The engine knows ONLY the contract. It has never heard of any concrete
7     // sink.
8     public void settle(Payable[] payees, LedgerSink sink) {
9         long total = 0;
10        for (Payable p : payees) {
11            total += p.amountOwedThisMonth();
12            sink.write(p.receiptLine());
13        }
14        sink.write("TOTAL: " + total + " paisa");
15    }
16 }

```

Listing 10: Depend on the contract, not on any one implementation.

The concrete destinations are written separately, and none of them requires touching `PayrollRun`:

```

1 public class ConsoleSink implements LedgerSink {
2     public void write(String line) { System.out.println(line); }
3 }
4 public class FileSink implements LedgerSink {
5     private final StringBuilder buffer = new StringBuilder();
6     public void write(String line) { buffer.append(line).append('\n'); }
7     public String contents() { return buffer.toString(); }
8 }
9
10 // Same engine, different behaviour chosen by what we hand in:
11 PayrollRun run = new PayrollRun();
12 run.settle(payees, new ConsoleSink()); // prints to screen
13 run.settle(payees, new FileSink());    // accumulates into a file buffer

```

Listing 11: Swap implementations freely; the engine never changes.

Three properties follow, and they are the reason interfaces sit at the centre of good design:

- **The user depends only on the promise.** `PayrollRun` names the type `LedgerSink` and nothing more concrete. It cannot accidentally rely on a detail of `ConsoleSink`, because it cannot *see* one. The contract is the entire surface between them.
- **Implementations are swappable.** Any present or *future* class that implements `LedgerSink` — a database sink written next year — drops straight in, and `PayrollRun` is not recompiled or even reopened. The set of implementers is open; the contract is closed.
- **The two sides can be built and tested independently.** You can test `PayrollRun` with a tiny fake `LedgerSink` that just records what it was asked to write, long before any real destination exists.

This is the design maxim often stated as “*program to an interface, not an implementation.*” The interface is a seam: a deliberate, narrow line drawn through the system at which one side states *what* it needs and the other side supplies *how*, with neither side able to reach past the seam

into the other's internals. Encapsulation hid the internals of a single object behind its methods; abstraction through interfaces hides an entire *component* behind a contract.

Teaching Insight

Make the seam physical. Run `settle` once with `ConsoleSink` and once with `FileSink` in front of the class, then state the punchline: *not one character of `PayrollRun` changed between the two runs, and it would not change for a database sink either*. That invariance — the using code frozen while the provided behaviour varies — is what abstraction buys, and it is visible on the projector in two lines.

Question to Carry Forward (Q3)

`PayrollRun.settle` takes its destination as a *parameter* of the interface type `LedgerSink`, rather than doing `new ConsoleSink()` inside itself. Both would print the same output today. But suppose next year you must send the ledger to a database instead. In which version do you edit `PayrollRun`, and in which do you leave it untouched and simply hand in a new object? What does that say about *where* a design should decide which concrete class to use?

Clue: the class that writes `new ConsoleSink()` is the class that *depends on* `ConsoleSink`. Follow which class is forced to change, and you have found where the dependency really lives.

11 Synthesis and Classroom Discussion

By the end of the development, the class should have internalised the following.

1. An interface is a *contract* — a named set of method promises, with no per-object state and no constructor. Its methods are implicitly `public abstract`; its fields are implicitly `public static final` constants.
2. `implements` makes the class a *subtype* of the interface, delivering the same runtime polymorphism as `extends` — one variable, many implementers, late dispatch by the JVM — but *without* inheriting any code.
3. A class may implement *many* interfaces (roles compose because promises never collide) but may `extend` only *one* class (state and code would collide).
4. Interfaces group objects by what they *can do*, cutting across unrelated hierarchies; abstract classes group by what things *are*, within one family.
5. Choose an abstract class for a true *is-a* family with shared state and code; choose an interface for an *ability* any class might have, or when a class must play several roles. The best designs use both, on the same class.
6. Modern interfaces may also hold `default` methods (to evolve a contract without breaking implementers), `static/private` helpers, constants, and — when they have a single abstract method — be implemented by a lambda. Conflicting `default` bodies must be resolved by the implementing class, because *behaviour* is what creates ambiguity.
7. Programming *to* an interface abstracts a whole component behind a contract: the user depends only on the promise, implementations are swappable without touching the user, and the two sides can be built and tested apart. This is the seam at the heart of maintainable design.

A productive closing exercise: hand students a class that already lives in an inheritance hierarchy — say the **Shape** family from the drawing lab — and a new cross-cutting requirement (“any shape that can be saved to disk,” “anything that can be drawn on a canvas,” “anything that can be compared by area”). Ask them to decide, for each requirement, whether it belongs in the class hierarchy or in a new interface, to name the interface as an *ability*, and to identify one *unrelated* non-shape class that could honestly implement the same interface. The moment they place a **Comparable** on both a **Circle** and an **Employee**, they have felt the thing an abstract class could never have done.

We now have two ways to create the subtypes that polymorphism feeds on — inheriting an implementation with **extends**, or promising a contract with **implements** — and a principled way to choose between them. Together with encapsulation (hiding an object’s internals) and abstraction (hiding a component’s internals behind a contract), the four pillars are in place. What remains is learning to combine them under pressure: real systems where several capabilities, several hierarchies, and several implementers must coexist cleanly — which is where the labs go next.
